

LLM Agent System with Secure Multi-Source Retrieval

Project Title:

"Design and Implement an Agentic System for Secure Multi-Source Question Answering"

Objective

Build a **conversational agent system** capable of answering user questions using information extracted from:

- Multiple long-form **PDF documents** (up to 500 pages each, dozens in total)
- A set of **external websites**, some of which may be protected by **anti-bot firewalls** (e.g., Cloudflare)

The agent should intelligently retrieve, reason, and respond using these sources, while strictly enforcing **Role-Based Access Control (RBAC)** and secure access via **JWT authentication**.

System Requirements

1. JWT-Based Access Control

- Users authenticate via a simulated login and receive a **JWT** containing their role.
- All backend APIs must verify JWTs and enforce permissions based on the encoded role.

2. Role-Based Access Matrix

Role	PDF Access	Web Access
Manager	✓	✓

Assistant Manager	✓	✗
Developer	✗	✓

- The system must ensure users can **only use the retrieval tools permitted for their role**.
- Role selection should be enabled via a **dropdown in the frontend**.

3. PDF Retrieval Module

- Support for **uploading and querying 500-page PDFs**.
- The system should chunk and index documents efficiently for retrieval.
- Answers must include **citations** (PDF title and page number).
- PDFs should be dynamically added or removed without requiring a code update.

4. Website Retrieval Module

- The system should accept a dynamic list of external URLs for scraping.
- Scrapers must handle **advanced protection mechanisms**, including:
 - Cloudflare (JavaScript-based firewalls, captchas, bot detection)
 - User-Agent rotation
 - Headless browser rendering (e.g., Playwright, Puppeteer)
 - Optional support for third-party anti-bot APIs (e.g., ScraperAPI, Zyte, CapSolver)
- Website content should be preprocessed, stored, and indexed for retrieval.
- Answers must cite the **source URL** and relevant snippet.

5. Agent Orchestration

- A unified agent (using LangChain or similar framework) must:
 - Handle user queries
 - Decide which tool to invoke (PDF or Web)
 - Respect RBAC constraints (only allowed tools per role)
 - Return a final answer with proper source attribution

6. Frontend Interface

- Include a simple interface (e.g., Streamlit or React):
 - Role selection dropdown (Manager, Assistant Manager, Developer)
 - JWT generation and injection
 - Query input + chat history
 - Upload PDFs and add website URLs
 - Display answers with source citations

Expected Features

- Dynamic upload of PDFs and website URLs
- Properly attributed responses (page number or URL)
- Secure access control using JWTs
- Agent-based orchestration with reasoning
- Robust scraping infrastructure for real-world web pages

Evaluation Focus

- Ability to design a modular, extensible system
- Handling of real-world complexity in scraping and retrieval

- Use of industry-standard libraries and practices (LangChain, FAISS, JWT)
- Secure architecture (RBAC, input sanitization)
- Clean and professional code structure

Deliverables

You are not expected to implement every detail. Instead:

1. **Design the overall system architecture** and folder/module structure.
2. **Implement at least one agent tool** (PDF or Web) end-to-end with citation support.
3. **Demonstrate RBAC enforcement** with JWT.
4. **Document how dynamic uploads are handled**, even if simulated.
5. Bonus: Add a UI to simulate role switching and query execution.

Final Note for the Candidate

This task is intentionally complex and open-ended. You are encouraged to:

- Make thoughtful architectural decisions
- Consider scalability and edge cases
- Justify your trade-offs clearly in documentation or comments
- Although the paid APIs are allowed, usage of open source models would be a plus